the existing file, I wrapped the specs that were already there in a describe block and added this code as a second describe block:

basics/05/spec/models/project_spec.rb

```
describe "estimates" do
  let(:project) { Project.new }
  let(:done) { Task.new(size: 2, completed: true) }
  let(:small_not_done) { Task.new(size: 1) }
  let(:large_not_done) { Task.new(size: 4) }

  before(:example) do
    project.tasks = [done, small_not_done, large_not_done]
  end
  it "can calculate total size" do
    expect(project.total_size).to eq(7)
  end
  it "can calculate remaining size" do
    expect(project.remaining_size).to eq(5)
  end
end
```

You put the spec in a new describe block in your project_spec.rb file since it has a different setup from the specs you've already written. Each describe block can have its own setup, making it easier to see what setup goes with what spec.

The let statements combined with the before block set up a project with three tasks; the two single-line it blocks have this setup in common. In RSpec, anything in a before(:each) or before(:example) block is executed as part of the setup before each spec.

These specs follow a common RSpec pattern: The "given" data goes in a series of let methods, the "when" action goes in a before block, and then a series of small it statements represent individual assertions.

A couple of minor style choices make the test easier to manage. All the task objects have meaningful names so that at a glance I can tell each object's reason for being in the test. If the tasks had descriptions or names I'd also give them meaningful data so that if the object gets printed to the terminal it's easy to tell which object it is. The specific score numbers that I'm using for each are also a deliberate style choice. Each task has a different score, and neither of the two adds up to the third, which is a very small thing that makes it harder to get a false positive test.

Prescription 6	Choose your test data and test-variable names to make it easy to diagnose failures when they happen. Meaningful names and data that doesn't overlap are helpful.